

Frameworks e Interoperabilidad

Trabajo Práctico N°2: CMS y Frameworks Web

Profesores:

Marcos Cruz
Jorge Navarro
Lara Acuña Bravo

Integrantes

Lucas San Segundo — FAI 1921
Joaquín Ignacio Castillo — FAI 5521



Lucas San Segundo



Joaquín Ignacio Castillo

Octubre 2026

Índice

Índice	2
Introducción	3
Dominio del proyecto: JS Computación	3
Repositorios y aclaración sobre el proyecto	3
Marco teórico	3
¿Qué es un CMS?	3
¿Qué es un framework?	4
Elección del CMS y de los frameworks	4
CMS: WordPress frente a Wix	4
Frontend: React frente a Preact	6
Backend: Express frente a Fastify	8
Template y modificaciones	9
Entorno de trabajo	9
Template elegido en WordPress	9
Cambios realizados	11
Módulos	11
Módulo 1: Página de inicio	11
Módulo 2: Pantalla de login	14
Módulo 3: Catálogo de productos	16
Conclusión	18

Introducción

El presente informe expone la resolución del Trabajo Práctico N°2 de Frameworks e Interoperabilidad. En el transcurso del informe se desarrolla la elección de un CMS y de frameworks web aplicados al dominio del Trabajo Final, la modificación de un template y la creación de módulos.

Dominio del proyecto: JS Computación

JS Computación es un comercio ubicado en Allen, Río Negro, dedicado a la venta de computadoras, insumos y servicios informáticos. Por eso, todas las decisiones se tomaron pensando en un e-commerce que cuenta con un catálogo de productos que puede crecer, un carrito de compras, cobro mediante Mercado Pago y un volumen de tráfico propio de un comercio local.

Repositorios y aclaración sobre el proyecto

El Trabajo Práctico N°2 está desarrollado en el repositorio https://github.com/Lucasanse/TP2_FI_JSComputacion_Proyect. La versión de React que se encuentra allí es estática, ya que en este práctico no se agrega interoperabilidad con otros sistemas.

El proyecto real del Trabajo Final se encuentra en los repositorios <https://github.com/Lucasanse/JS-Proyect-FE> (frontend) y <https://github.com/Lucasanse/JS-Proyect-BE/> (backend).

Marco teórico

¿Qué es un CMS?

Antes de iniciar el desarrollo del informe es importante mencionar el marco teórico de lo que se evaluará a continuación. Un Sistema de Gestión de Contenidos (CMS, por sus siglas en inglés) es un software que facilita la gestión de sitios web y permite a administradores, editores y otros usuarios crear, clasificar y publicar información sin necesidad de saber programar. El contenido se almacena en una base de datos y se maneja de forma independiente del diseño [11].

Todo CMS debe cubrir cuatro funciones esenciales: creación, gestión, publicación y presentación del contenido. Se compone de un back-end, donde los administradores cargan el contenido, y un front-end, donde los visitantes lo ven generado dinámicamente. Para elegirlo, la cátedra propone criterios como arquitectura, grado de desarrollo, soporte, posición en el mercado, usabilidad, accesibilidad, seguridad, velocidad y funcionalidades, que son los que se usaron en la comparación de este informe [11].

¿Qué es un framework?

Un framework es una estructura de soporte sobre la cual se organiza y desarrolla otro proyecto de software; puede pensarse como una aplicación genérica, incompleta y configurable a la que se le agregan las últimas piezas para construir una aplicación concreta [12]. Su objetivo es resolver funcionalidades comunes (sesiones, acceso a datos, validaciones), acelerar el desarrollo reutilizando código y promover buenas prácticas y código organizado. Como contraparte, puede sumar sobrecarga de recursos, dependencia del código del framework y tiempo de aprendizaje [12]. Se distinguen frameworks del lado del cliente y del lado del servidor, entre ellos los frameworks para Node.js como Express.

También conviene diferenciar framework de librería: una librería ofrece funciones que se usan a elección, mientras que un framework impone el flujo de trabajo [12]. En ese sentido, React es técnicamente una librería de interfaces, aunque en la práctica se la trata como framework de frontend.

Elección del CMS y de los frameworks

Para cada capa del proyecto se compararon dos herramientas similares y se eligió una de ellas. Para que la comparación sea justa se utilizaron los mismos criterios en cada caso, y se puso especial atención en las características negativas de cada alternativa, ya que fueron estas las que terminaron de inclinar la balanza.

CMS: WordPress frente a Wix

En el caso del **CMS** se compararon WordPress y Wix. WordPress es un CMS de código abierto y autoalojado y Wix es una plataforma propietaria que funciona como un servicio “todo en uno” (constructor visual, hosting y CMS en un mismo producto).

Criterio	WordPress	Wix
Facilidad de uso	Media. Requiere instalar, configurar hosting, temas y plugins. Con constructores como Elementor se vuelve visual.	Alta. Editor drag & drop y asistente con IA; se publica sin conocimientos técnicos.
Escalabilidad	Alta. Se puede migrar a un mejor hosting, usar caché, CDN o modo headless. El límite lo pone la infraestructura.	Media. Limitada por el plan contratado (almacenamiento, productos, colaboradores) y por la plataforma.
Extensiones	Más de 60.000 plugins y miles de temas en el repositorio oficial, más los comerciales.	Wix App Market con algunos cientos de apps; solo se usan las que Wix aprueba.
Precios	Software gratuito. Se paga hosting, dominio y, opcionalmente, temas o plugins premium.	Plan gratis con publicidad y subdominio. Planes pagos desde ~USD 17/mes [4].

Criterio	WordPress	Wix
Arquitectura	PHP + MySQL/MariaDB, autoalojado. Monolítico, pero puede exponer una API REST (headless).	SaaS cerrado alojado por Wix. Permite algo de código propio con Velo (JavaScript).
Posicionamiento	Líder: ~43% de todos los sitios web y ~60% de los sitios con CMS conocido [1][2].	Minoritario: ~4% de todos los sitios y ~5-6% de los sitios con CMS [1][2].
Usabilidad	Panel potente pero con muchas opciones; depende del tema y los plugins elegidos.	Interfaz unificada y guiada; poca curva para el usuario final.
Accesibilidad	Depende del tema: existen temas "accessibility-ready" y plugins de ayuda. Controlable a nivel de código.	Herramientas integradas, pero sin control total del HTML generado.
Seguridad	Responsabilidad del administrador: actualizar núcleo y plugins. Los plugins desactualizados son el principal riesgo.	Gestionada por Wix: SSL, parches y servidores administrados. Menos control, menos mantenimiento.
Velocidad de carga	Variable: depende del hosting, el tema, los plugins y la optimización. Puede ser muy rápida bien configurada.	Aceptable y estable con CDN incluida, pero poco optimizable: carga scripts propios de la plataforma.
Integración con Mercado Pago	Sí. Plugin oficial y gratuito para WooCommerce, con Checkout Pro o Checkout API [3].	Sí, sin programar, en los planes con e-commerce; comisión desde 3,99% por transacción [5].

Puntos débiles de Wix que motivaron su descarte

Wix es una muy buena opción para quien no tiene conocimientos de desarrollo web y necesita publicar rápido; si ese fuera nuestro caso, probablemente la elegiríamos. Sin embargo, para un e-commerce que pretende crecer y para un equipo que sí programa, las tres desventajas más importantes fueron:

- **Tiene dependencia de la plataforma.** El sitio vive dentro de Wix: no se puede llevar a otro servidor ni acceder al código del lado del servidor. Si en el futuro cambian los precios o las condiciones, la única salida es rehacer el sitio en otra herramienta.
- **Una vez publicado el sitio, el template no se puede cambiar.** Para hacerlo hay que crear un sitio nuevo con otro template y migrar el contenido a mano [9]. Esto choca incluso con lo pedido en este práctico, donde se busca partir de un template y modificarlo con libertad.
- **Es limitado por el plan.** La cantidad de productos, el almacenamiento y las funciones de e-commerce dependen del plan que se pague. El catálogo de un comercio de informática (equipos, componentes, periféricos, insumos) crece rápido, y cada salto de plan es un costo mensual en dólares.

¿Por qué elegimos WordPress?

Teniendo en cuenta lo anterior, elegimos WordPress para JS Computación por los siguientes motivos:

- **Por su escalabilidad sin techo de plan.** El catálogo puede crecer sin depender de los límites de un plan y en caso de aumentar el tráfico, se mejora el hosting.
- **El software es gratuito.** Se paga hosting y dominio, que a largo plazo suele salir menos que un plan de e-commerce de Wix.
- **El plugin WooCommerce** cuenta con el plugin oficial y gratuito de Mercado Pago para Argentina [3], además de plugins de cálculo de envíos y facturación.
- **Se puede cambiar de tema en cualquier momento** y permite crear un tema hijo para modificar estilos sin perder las actualizaciones, lo que da flexibilidad para modificar el template.
- **Abunda la documentación y es una habilidad muy demandada** al ser el CMS más usado del mundo [1].

La principal desventaja que asumimos es que la seguridad y el mantenimiento quedan a nuestro cargo: hay que mantener actualizados el núcleo, el tema y los plugins, y elegir con cuidado qué plugins se instalan.

Frontend: React frente a Preact

Para el **frontend** se compararon React y Preact. Preact ofrece casi la misma API que React en una fracción del tamaño; React ofrece el ecosistema más grande y las funciones más nuevas.

Criterio	React	Preact
Tipo	Biblioteca de JavaScript para construir interfaces de usuario (no es un framework completo).	Biblioteca alternativa a React con API compatible (es otro proyecto, no una versión de React).
Lenguaje principal	JavaScript con JSX; soporte completo para TypeScript.	JavaScript con JSX; soporte para TypeScript.
Curva de aprendizaje	Media: componentes, props, estado, hooks. Mucho material y cursos.	Media-baja si ya se sabe React; menos documentación y tutoriales propios.
Arquitectura	Componentes, DOM virtual con reconciliador "Fiber", renderizado concurrente, Server Components [6].	Componentes, DOM virtual más simple y liviano; Signals para estado reactivo; eventos nativos del navegador [6][7].
Escalabilidad	Muy alta: usado en aplicaciones enormes (Meta, Netflix, Airbnb) y con frameworks como Next.js.	Alta, pero con menos herramientas y casos probados en proyectos muy grandes.
Extensibilidad	El ecosistema más grande: React Router, Redux, Zustand, TanStack Query, bibliotecas de componentes.	Usa la mayoría de las librerías de React mediante preact/compat, aunque algunas pueden fallar o requerir ajustes [6][7].

Criterio	React	Preact
Casos de uso	Aplicaciones web complejas, SPA, e-commerce, dashboards, apps móviles con React Native [6].	Sitios donde el peso es crítico: widgets embebidos, PWA, landing pages [6].
Usabilidad	Documentación oficial completa, React DevTools, errores claros y soporte en todas las herramientas (Vite, Next.js).	Buena: misma sintaxis que React y Preact DevTools, pero documentación más acotada y menos ejemplos.
Accesibilidad	Tiene el ecosistema de accesibilidad más amplio: eslint-plugin-jsx-a11y para detectar errores al programar, librerías de componentes accesibles como React Aria, Radix UI o Headless UI, y herramientas de auditoría como axe-core.	Su ecosistema propio es más chico. Para usar librerías de componentes accesibles pensadas para React (React Aria, Radix UI) depende de preact/compat, con posibles incompatibilidades. eslint-plugin-jsx-a11y funciona igual, ya que analiza JSX.

Puntos débiles de Preact que motivaron su descarte

Como virtud, Preact es mucho más chico, lo que lo hace ideal cuando el peso y la velocidad de arranque son lo más importante. Sin embargo, para nuestro caso las tres desventajas más importantes fueron:

- Para usar librerías del ecosistema React hay que pasar por la capa preact/compat, y algunas pueden fallar o necesitar ajustes [7]. Un e-commerce depende justamente de varias de esas librerías (ruteo, manejo de estado del carrito, formularios, componentes de interfaz).
- Preact sigue la API de React, pero no implementa todo lo que React incorpora (por ejemplo, partes del renderizado concurrente o los Server Components), por lo que el proyecto quedaría atado a lo que Preact logre replicar [6][7].
- Hay mucha menos documentación, tutoriales y respuestas en foros específicos de Preact. Ante un error, la mayoría de las soluciones disponibles están pensadas para React.

¿Por qué elegimos React?

- El e-commerce necesita ruteo, manejo de estado del carrito, formularios y pagos, y React tiene librerías probadas para todo eso sin capas de compatibilidad.
- Es la herramienta que venimos usando en la carrera junto con Tailwind CSS, lo que nos permite especializarnos en lugar de dispersarnos.
- Es la librería de frontend más usada del mercado y la más pedida en búsquedas laborales.
- Como vamos a utilizar un catálogo de productos con carrito, es una aplicación interactiva, el caso de uso para el que React está optimizado.

Backend: Express frente a Fastify

Para el **backend** se compararon Express y Fastify, dos frameworks web para Node.js. Fastify es más rápido y trae validación, logging y tipos de TypeScript integrados; Express es más simple y tiene la comunidad más grande de Node.js.

Criterio	Express	Fastify
Tipo	Framework web minimalista para Node.js [10].	Framework web para Node.js enfocado en rendimiento y bajo overhead [8].
Arquitectura	Cadena de middlewares (req, res, next) y router.	Sistema de plugins encapsulados, hooks del ciclo de vida de la petición y decoradores [8].
Rendimiento	Bueno para la mayoría de los proyectos; más lento en benchmarks [8].	Uno de los más rápidos de Node.js: en el benchmark oficial maneja aprox. 1,5 veces más peticiones por segundo que Express 5 [8].
Validación de datos	No incluida: se agregan librerías como Zod, Joi o express-validator.	Integrada con JSON Schema para validar entradas y serializar respuestas [8].
Curva de aprendizaje	Baja. Pocas reglas y muchísimos ejemplos.	Media. Hay que entender plugins, encapsulación y schemas.
Comunidad y ecosistema	La más grande de Node.js: miles de middlewares y respuestas en foros.	Más chica pero activa; ecosistema oficial de plugins. Los middlewares de Express requieren un adaptador (@fastify/express).
Integración con otras herramientas	Compatible con casi todo: Prisma, Passport, JWT, Multer, etc. NestJS lo usa por defecto.	Compatible con Prisma, JWT, etc. mediante plugins.
Casos de uso	APIs REST chicas o medianas, prototipos, proyectos educativos, equipos que priorizan simplicidad.	APIs con mucho tráfico, microservicios, proyectos donde rendimiento y tipado estricto son prioridad.

Puntos débiles de Fastify que motivaron su descarte

Reconocemos que Fastify tiene ventajas técnicas reales: es más rápido, trae validación integrada y sus tipos de TypeScript vienen incluidos, algo que Express no ofrece de forma nativa. Aun así, lo descartamos por estos tres motivos:

- La principal ventaja no es determinante en nuestro proyecto. La diferencia de rendimiento importa en APIs con muchísimo tráfico; para el volumen de un comercio local no es un factor decisivo, y el cuello de botella va a estar en la base de datos y no en el framework.
- Su modelo de plugins encapsulados, hooks, decoradores y schemas obliga a aprender una forma de trabajo nueva, cuando el equipo está aprendiendo el modelo de middlewares de Express y especializándose en él.

- Hay menos ejemplos, tutoriales (especialmente en español) y respuestas en foros.

También se consideró **NestJS**, un framework más completo y estructurado. Se descartó porque impone una arquitectura propia (módulos, decoradores, inyección de dependencias) y TypeScript como base, lo que agrega complejidad innecesaria para el tamaño de este proyecto y se aleja de lo que venimos trabajando en la carrera.

¿Por qué elegimos Express?

- Es lo que venimos viendo en la carrera y nos permite especializarnos en una herramienta en particular.
- Funciona muy bien con Node.js, Prisma y PostgreSQL, y hay abundante documentación y videos de esa combinación, que es el stack del Trabajo Final.
- Ante cualquier error hay miles de ejemplos, respuestas y resoluciones disponibles.
- Lo que no trae de fábrica (validación, tipado) se puede incorporar con librerías conocidas, sin cambiar de framework.

Template y modificaciones

Entorno de trabajo

En un principio se había pensado alojar el sitio en InfinityFree, un hosting con plan gratuito para WordPress, pero finalmente no se utilizó. El sitio en WordPress se trabajó de forma local con “Local”, una aplicación de escritorio que permite crear e instalar sitios WordPress en la propia computadora sin necesidad de un hosting.

Para los estilos se evaluó usar Elementor, un plugin constructor de páginas que facilita la edición visual y ofrece plantillas muy llamativas. Sin embargo, en su versión gratuita está orientado a sitios con poca funcionalidad (blogs, sitios institucionales), y varias de las herramientas que necesitaría un e-commerce son pagas. Por eso se trabajó con el editor de plantillas que trae WordPress, que ya brinda una base sobre la cual empezar.

La versión en React se desarrolló con Vite como herramienta de build, TypeScript y Tailwind CSS. Se eligió Vite en lugar de Create React App, que está discontinuado. Para este informe se tomó el frontend del Trabajo Final y se adaptó a una versión estática con las páginas Inicio, Productos y Login. Los productos se cargan desde un archivo local (src/data/productos.ts) en lugar de la API del backend, por lo que se puede ejecutar sin levantar el servidor ni la base de datos.

Template elegido en WordPress

Se utilizó el tema por defecto de WordPress, editado con la herramienta de edición de plantillas del propio WordPress. Se ajusta al dominio de JS Computación porque genera páginas responsive sin configuración adicional y es compatible con WooCommerce, que se usó para cargar y mostrar los productos. A continuación se muestra el template tal como se ve antes de cualquier modificación, para poder compararlo luego con la solución propuesta.

WordPress con el tema por defecto (escritorio y mobile)



Como en React no se partió de un template, no hay un estado inicial para mostrar ya que los componentes se desarrollaron directamente con la paleta y los estilos del proyecto. Su resultado se muestra en la sección Módulos.

Cambios realizados

Las decisiones tomadas durante el desarrollo (herramientas, plugins y cambios de diseño), con su fecha y su motivo, se encuentran registradas en el archivo decisiones.md del repositorio

https://github.com/Lucasanse/TP2_FI_JSComputacion_Proyect/blob/main/docs/decisiones.md.

En React, al no partir de un template, más que modificaciones se tomaron decisiones de desarrollo. Las principales fueron:

- Se extrajo el frontend del Trabajo Final que teníamos realizado hasta ahora y se adaptó a una versión estática (Inicio, Productos y Login) para enviar en este trabajo práctico.
- Se guardaron imágenes dentro del proyecto para exhibir en este trabajo práctico.
- En la página de inicio se rediseñó el banner que tenía la versión anterior por uno oscuro con los colores del logo, un texto orientado a la venta de productos y tarjetas de Armado de PC y Servicio técnico. Los productos destacados se muestran al azar en cada visita. A medida que avance el desarrollo del trabajo práctico final, se remodelarán varias de estas funcionalidades.
- La paleta de colores se definió en el @theme de Tailwind CSS para usarla de forma consistente en todos los componentes.

En WordPress, como el template original no se ajustaba a lo que buscábamos, se renovó casi toda la interfaz. En la misma se aplicó la paleta del logo, se agregó una barra de navegación y se rediseñaron la página de inicio, el catálogo y el login. El detalle de cada página se describe en la sección Módulos.

La identidad visual del sitio parte del logo de JS Computación, del cual se tomaron sus dos colores principales: el bordó y el blanco con un matiz celeste. Los colores se pensaron verificando su relación de contraste según las pautas WCAG 2.1, que exigen un mínimo de 4,5:1 para texto normal (nivel AA) y de 3:1 para texto grande y elementos gráficos.

Respecto a la tipografía, en la versión en React se utiliza la fuente del sistema operativo (system-ui, con Segoe UI y Roboto como alternativas). Esto evita descargar fuentes externas, mejora la velocidad de carga y le da al usuario una tipografía que ya le resulta familiar y legible. Esta elección puede variar a lo largo del desarrollo del trabajo práctico final, teniendo en cuenta que estamos en una etapa temprana de desarrollo.

Módulos

Se crearon y/o modificaron al menos tres módulos. Para cada uno se describen sus características y funcionalidad, junto con los plugins, librerías u otras herramientas utilizadas para lograrlas.

Los módulos son la página de inicio, la pantalla de login y el catálogo de productos, y cada uno se desarrolló tanto en WordPress como en React. En cada módulo se muestran las capturas de ambas versiones.

Módulo 1: Página de inicio

Implementación en Wordpress (escritorio y mobile)


[Productos](#)
[Armá tu PC](#)
[Servicio técnico](#)



[Notificaciones](#)

TIENDA DE COMPUTACION Y MAS!

Deja de buscar por todos lados, acá tenes todo

Una casa de tecnologia con todo lo que buscas

[Mirá nuestros productos](#)



Mirá

Productos destacados

[View all →](#)







TIENDA DE COMPUTACION Y MAS!

Deja de buscar por todos lados, acá tenes todo

Una casa de tecnologia con todo lo que buscas

[Mirá nuestros productos](#)

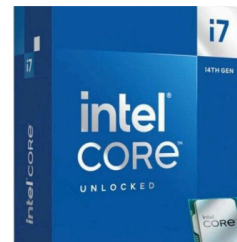
Productos destacados



Corsair Vengeance

~~\$ 60.000,00~~ \$ 50.000,00

[Agregar al carrito](#)

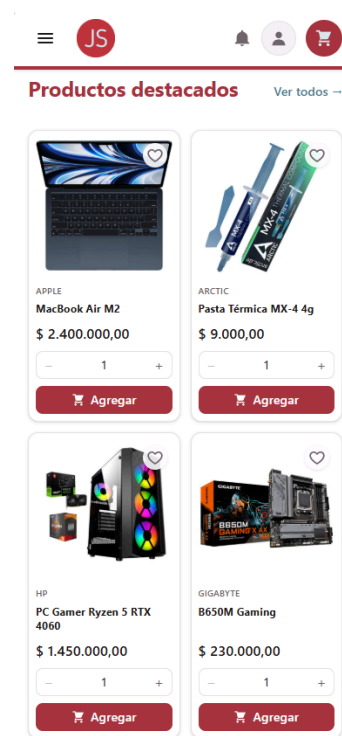
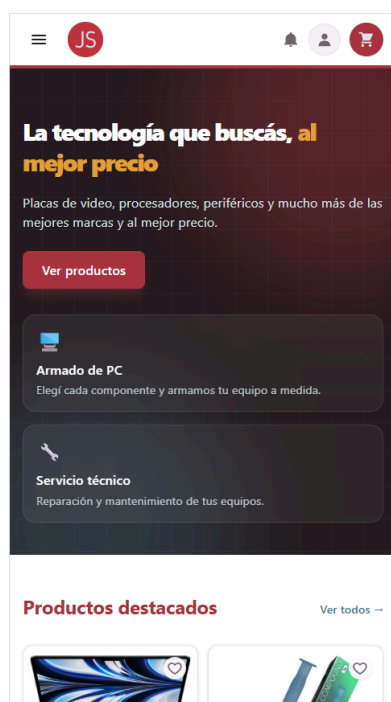
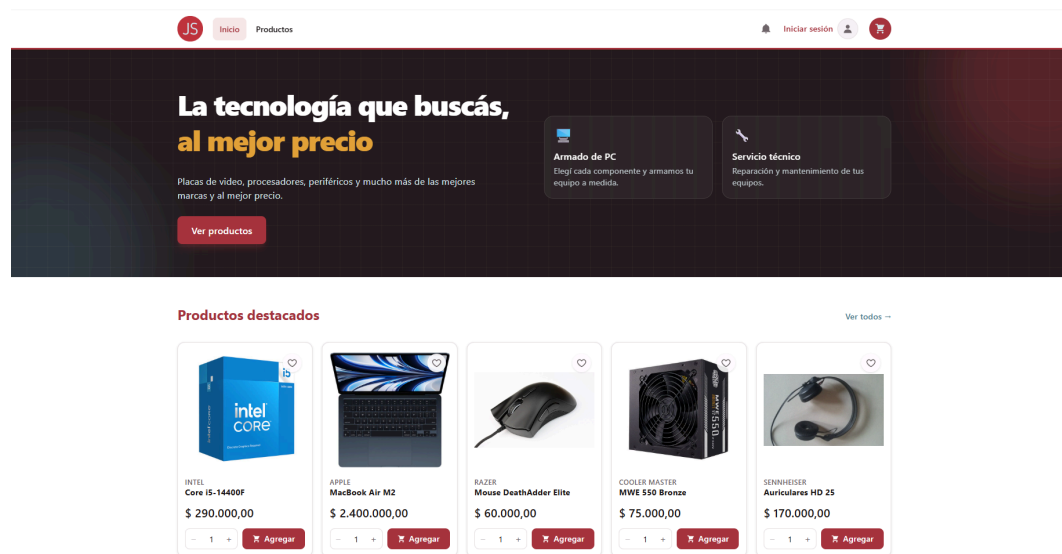


Intel Core i5-14400F

~~\$ 300.000,00~~
\$ 290.000,00

[Agregar al carrito](#)

Implementación en React (escritorio y mobile)



En esta página aplicamos las reglas de color del proyecto principal, basadas en la paleta del logo de JS Computación. También agregamos algunos productos destacados y estilos que hacen la página más atractiva. En WordPress tuvimos que renovar casi toda la interfaz, ya que el template original no se ajustaba a lo que buscábamos.

También agregamos una barra de navegación con las secciones que requiere el dominio. Además, se destaca la facilidad con la que WordPress genera páginas totalmente responsive, sin necesidad de hacer ajustes adicionales.

Módulo 2: Pantalla de login

Implementación en Wordpress (escritorio y mobile)


[Productos](#)
[Armá tu PC](#)
[Servicio técnico](#)





Notificaciones

Iniciar sesión

Nombre de usuario o correo electrónico *

Contraseña *

☐ Recordarme

¿Olvidaste la contraseña?



PRODUCTOS

RAM

Tarjetas graficas

Motherboards

SOBRE NOSOTROS

Informacion

Contacto









Iniciar sesión

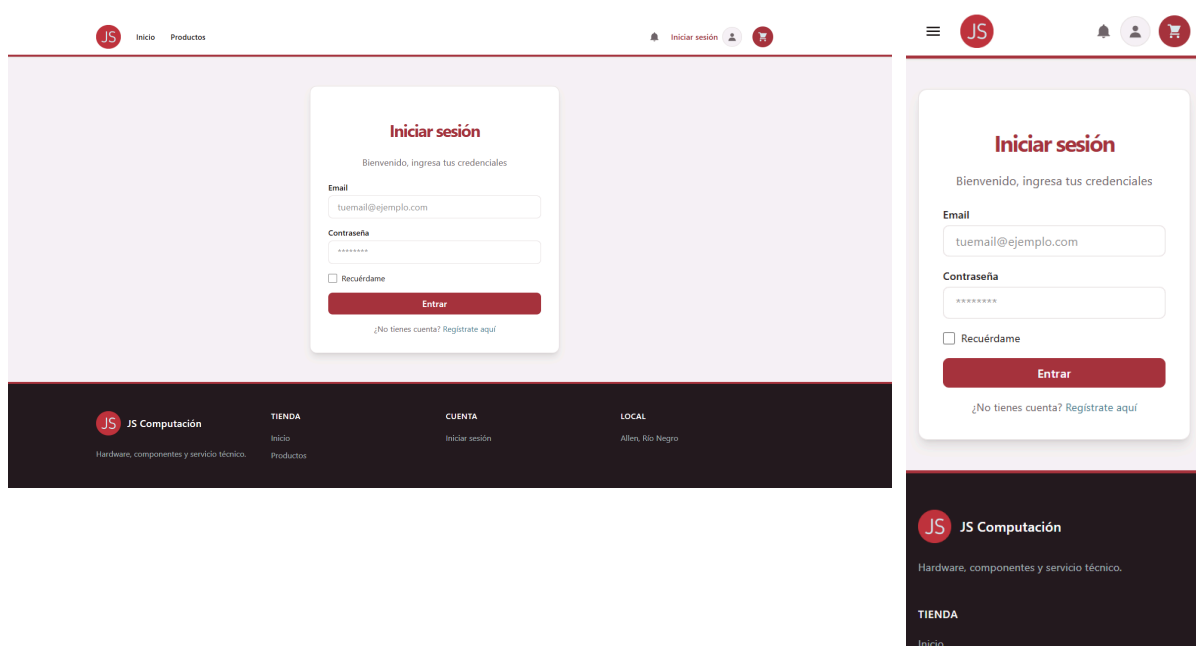
Nombre de usuario o correo electrónico *

Contraseña *



☐ Recordarme

Implementación en React (escritorio y mobile)



En WordPress, para la pantalla de login volvimos a utilizar los templates del propio editor, con algunos ajustes de color, y agregamos un footer acorde a la estética del sitio. Si bien es un login básico, se destaca la facilidad de la implementación ya que el propio editor de plantillas ofrece todas las herramientas necesarias para resolverlo sin recurrir a plugins adicionales.

En React, la pantalla de login se construyó como un componente propio, tomando como referencia el que ya teníamos en el frontend del Trabajo Final. Se aplicó la misma paleta de colores que en el resto del sitio, de modo que la pantalla mantiene su identidad visual. El footer, en tono oscuro, muestra el logo y los apartados Tienda, Cuenta y Local.

Módulo 3: Catálogo de productos

Implementación en Wordpress (escritorio y mobile)

Inicio / Productos

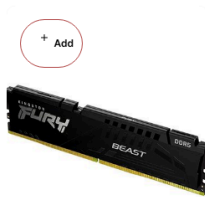
Productos

Tipo ▾



Z790 Tomahawk

\$ 230.000



Vengeance 16GB DDR4

\$ 50.000



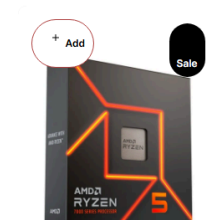
GeForce RTX 4070

\$ 480.000



Core i7-14700K

\$ 650.000



Ryzen 5 7600

\$ 320.000 ~~\$ 330.000~~

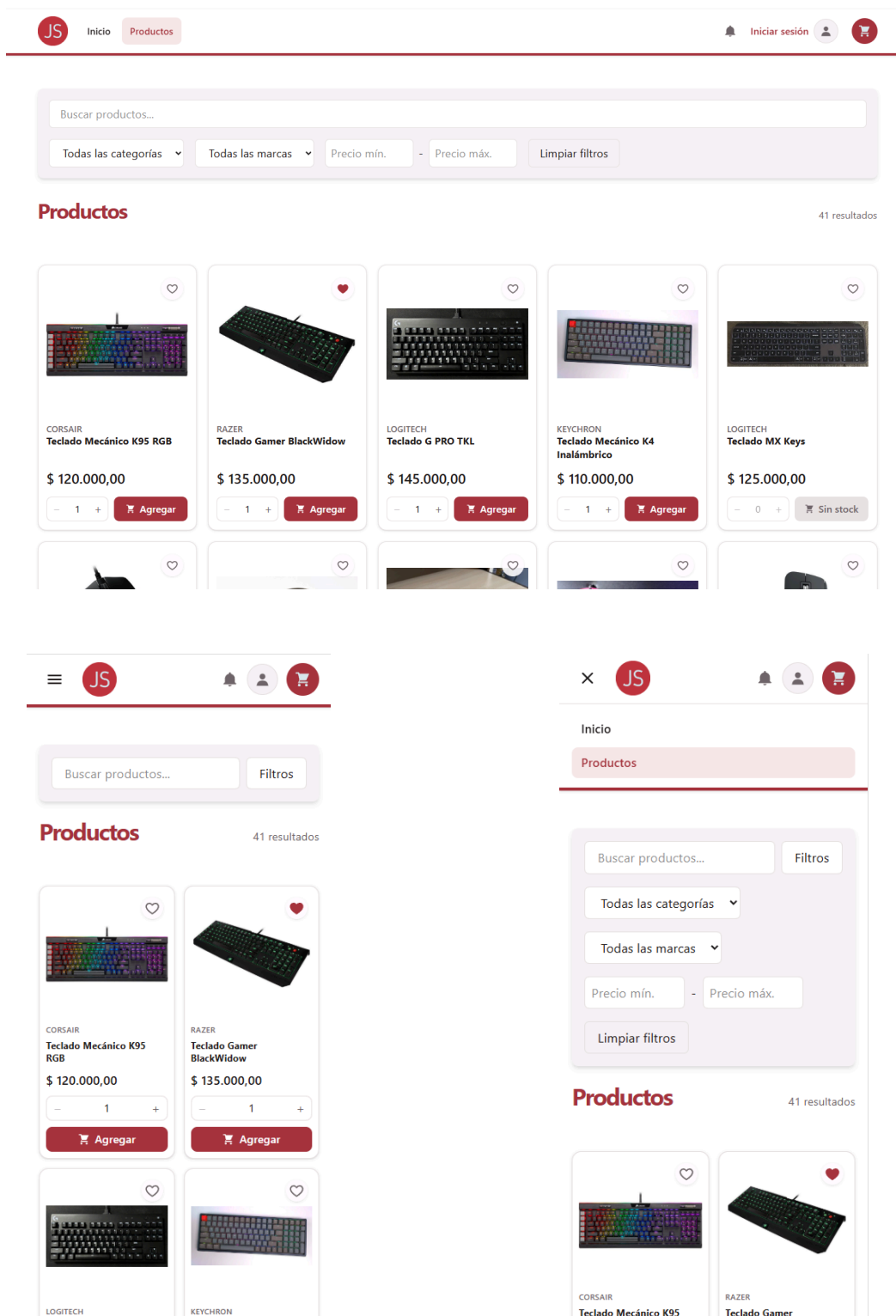


Z790 Tomahawk

\$ 230.000

Add to Cart

Implementación en React (escritorio y mobile)



En WordPress, la página del catálogo se desarrolló con su buscador. En esta etapa fue necesario incorporar plugins para cargar los productos y mostrarlos en pantalla. Todo lo relacionado con los productos se resolvió con WooCommerce y otros plugins de sus mismos creadores.

En React, los productos se cargan desde un archivo local con los mismos datos que usa el backend del Trabajo Final. En la parte superior se ubica el buscador, junto con los filtros por categoría, marca y rango de precio y un botón para limpiarlos. El filtrado se realiza directamente en el navegador, sin pedir datos a un servidor (a diferencia de lo implementado en el trabajo práctico final), y la página indica en todo momento la cantidad de resultados encontrados.

Cada producto se muestra en una tarjeta con su imagen, marca, nombre y precio. La tarjeta incluye además un selector de cantidad, el botón "Agregar" al carrito y un ícono para marcarlo como favorito. Cuando un producto no tiene stock, el botón se deshabilita y muestra "Sin stock". Este diseño fue motivado por trabajos prácticos anteriores que realizamos en el transcurso de la carrera.

Conclusión

En este trabajo hicimos un recorrido breve en torno a un mismo objetivo: construir una página web. Para ello exploramos distintas herramientas de backend, frontend y CMS, y analizamos las principales diferencias entre React y Preact y entre Express y Fastify.

Más allá de lo que indican los datos de mercado y las características técnicas de cada herramienta, nuestras elecciones respondieron principalmente a la comodidad y a la especialización. En el caso del CMS la decisión fue más indistinta, ya que ninguno de los dos había trabajado antes con uno, por lo que nos inclinamos por el líder del mercado.

Los CMS presentan una complejidad mucho menor que la programación tradicional, aunque siguen ofreciendo un amplio margen de personalización y de servicios. Por eso los recomendamos para alguien que se está iniciando en la programación. No los recomendamos, en cambio, para que alguien sin conocimientos de programación ni del entorno web encare un proyecto con funcionalidades más complejas, como un e-commerce que necesita crecer, ya que en ese caso el resultado sería pobre o nulo.

Por último, el trabajo de investigación nos permitió conocer herramientas que nunca habíamos utilizado, sobre todo los CMS, con los que no habíamos trabajado durante la carrera. Descubrimos que existen herramientas muy sencillas que permiten resolver problemas o desarrollar software sin necesidad de programar todo desde cero. En ese sentido, este trabajo nos abrió el panorama a una variedad de herramientas posibles para encarar futuros proyectos.

Bibliografía

- [1] Barn2 Media. (s.f.). *WordPress market share: How many websites use WordPress in 2026?* Recuperado el 26 de septiembre de 2026, de <https://barn2.com/blog/wordpress-market-share/>
- [2] BloggersPassion. (s.f.). *WordPress statistics (2026): Usage, growth trend & more.* Recuperado el 26 de septiembre de 2026, de <https://bloggerspassion.com/wordpress-security-tips>
- [3] Mercado Pago. (s.f.). *WooCommerce. Mercado Pago Developers.* Recuperado el 26 de septiembre de 2026, de <https://mercadopago.com.ar/developers/en/docs/woocommerce/installation>
- [4] Website Builder Expert. (s.f.). *Wix pricing 2026: Which plan should you pay for?* Recuperado el 26 de septiembre de 2026, de <https://www.websitebuilderexpert.com/wix-pricing>
- [5] Wix.com. (s.f.). *Conectar Mercado Pago como proveedor de pagos. Centro de Ayuda de Wix.* Recuperado el 26 de septiembre de 2026, de <https://support.wix.com/es/article/conectar-mercadopago-como-proveedor-de-pagos>
- [6] Yadav, P. (2022, 29 de noviembre). *Preact vs React: Which is faster? A practical performance guide.* Syncfusion Blogs. <https://www.syncfusion.com/blogs/post/preact-vs-react>
- [7] Preact. (s.f.). *Differences to React.* Recuperado el 26 de septiembre de 2026, de <https://preactjs.com/guide/v10/differences-to-react/>
- [8] Fastify. (s.f.). *Fastify: Fast and low overhead web framework, for Node.js.* Recuperado el 26 de septiembre de 2026, de <https://fastify.dev/>
- [9] TechRadar. (s.f.). *How to change a template on Wix.* Recuperado el 1 de octubre de 2026, de <https://www.techradar.com/pro/website-building/how-to-change-a-template-on-wix>
- [10] Express. (s.f.). *Express: Fast, unopinionated, minimalist web framework for Node.js.* Recuperado el 1 de octubre de 2026, de <https://expressjs.com/>
- [11] **Diapositivas Frameworks e Interoperabilidad. (2026).** *Unidad III: Desarrollos Web Abiertos - Sistemas gestores de contenido* [Diapositivas de clase]. Facultad de Informática, Universidad Nacional del Comahue.
- [12] **Diapositivas Frameworks e Interoperabilidad. (2026).** *Unidad III: Frameworks* [Diapositivas de clase]. Facultad de Informática, Universidad Nacional del Comahue.